FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Millares, Kurt Martin M.
Section:	BSCSSE - AN42
Professor:	Ms. Crisola Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   RBR is easier to implement because it only requires certain conditions to be satisfied in a given scenario to draw conclusions, make decisions, and perform actions according to predetermined solutions.
2. Which is more flexible in solving unstructured problems?
   CBR is more flexible because it can adapt solutions from past cases to handle new or unfamiliar problems, while RBR depends on fixed rules and conditions.
3. Which is better for situations with continuously changing data?
   RBR is more effective because it processes input data using predefined categories and variables. Even when the values change, they are still evaluated according to the same rules, with only the size or definition of the values varying.

4. Examples of real-world applications for both.\

In real-world applications, RBR is best suited for situations in which the variables and conditions can be evidently defined, and without much variance. For example, in GPS navigation and traffic management systems, factors such as traffic congestion, road conditions, and availability of alternative routes, can be empirically measured for an optimal or appropriate response. On the other hand, CBR is useful for complex problems with indeterminate or unidentifiable nuances and contexts, as it can utilize previous experiences to synthesize new solutions. For example, in legal practice, the persuasiveness of an argument can be strengthened by citing legal precedents, particularly cases in which courts have previously ruled in favor of a similar position and party.

Your Code - Laboratory Tasks

```
# ============================================
# PART 1 — KNOWLEDGE REPRESENTATION
# ============================================

# Facts will be filled in by the user
facts = {}


# ============================================
# PART 2 — RULE-BASED REASONING
# ============================================

# Task 2A — IF-THEN Rules

def rule_heavy_traffic(facts):
    if facts["traffic_level"] == "heavy":
        return "Take an alternate route"


def rule_rain(facts):
    if facts["rain"]:
        return "Drive carefully because the road is wet"


def rule_accident(facts):
    if facts["accident"]:
        return "Avoid the area because there is an accident"


def rule_construction(facts):
    if facts["road_construction"]:
        return "Use another road because of road construction"


# Store all rules
rules = [
```

```
    rule_heavy_traffic,
    rule_rain,
    rule_accident,
    rule_construction
]


# ============================================
# Task 2B — INFERENCE ENGINE
# ============================================

def inference_engine(facts, rules):

    actions = []

    for rule in rules:

        result = rule(facts)

        if result is not None:
            actions.append(result)

    return actions


# ============================================
# PART 3 — CASE-BASED REASONING
# ============================================

# Task 3A — Case Base

case_base = [

    {
        "problem": {
            "traffic_level": "heavy",
            "rain": False,
```

```
            "accident": True,
            "road_construction": False
        },
        "solution": "Take an alternate route to avoid the accident."
    },

    {
        "problem": {
            "traffic_level": "moderate",
            "rain": True,
            "accident": False,
            "road_construction": False
        },
        "solution": "Drive slowly and carefully because the road is wet."
    },

    {
        "problem": {
            "traffic_level": "heavy",
            "rain": True,
            "accident": False,
            "road_construction": True
        },
            "solution": "Use another road because of heavy traffic and
construction."
    }
]


# ================================================
# CBR — RETRIEVE
# ================================================

def similarity(current_problem, past_problem):

    score = 0
```

```
    for feature in current_problem:

        if current_problem[feature] == past_problem[feature]:
            score += 1

    return score


def retrieve(current_problem, case_base):

    best_case = None
    best_score = -1

    for case in case_base:

        score = similarity(
            current_problem,
            case["problem"]
        )

        if score > best_score:

            best_score = score
            best_case = case

    return best_case, best_score


# ============================================
# CBR — REUSE
# ============================================

def reuse(retrieved_case):

    if retrieved_case is not None:
        return retrieved_case["solution"]
```

```
    return "No similar case found."


# ================================================
# CBR — REVISE
# ================================================

def revise(solution):

    print("\nRetrieved Solution:")
    print(solution)

    new_solution = input(
        "\nEnter a new solution "
        "(or press Enter to keep the retrieved solution): "
    )

    if new_solution.strip() == "":
        return solution

    return new_solution


# ================================================
# CBR — RETAIN
# ================================================

def retain(case_base, problem, solution):

    new_case = {
        "problem": problem,
        "solution": solution
    }

    case_base.append(new_case)
```

```
# ============================================
# USER INPUT
# ============================================

print("====================================")
print("TRAFFIC REASONING SYSTEM")
print("====================================")

print("\nPlease enter the current traffic situation.\n")


# Get traffic information from the user

traffic_level = input(
    "Traffic level (light/moderate/heavy): "
).lower()

rain_input = input(
    "Is it raining? (yes/no): "
).lower()

accident_input = input(
    "Is there an accident? (yes/no): "
).lower()

construction_input = input(
    "Is there road construction? (yes/no): "
).lower()


# Convert yes/no answers to True or False

rain = rain_input == "yes"
accident = accident_input == "yes"
road_construction = construction_input == "yes"
```

```
# Create the Knowledge Base using user input

facts = {
    "traffic_level": traffic_level,
    "rain": rain,
    "accident": accident,
    "road_construction": road_construction
}


# ============================================
# PART 2C — RUN RULE-BASED REASONING
# ============================================

print("\n====================================")
print("RULE-BASED REASONING")
print("====================================")

print("\nCurrent Traffic Facts:")
print(facts)


# Run inference engine AFTER user input

actions = inference_engine(facts, rules)


print("\nRule-Based Reasoning Actions:")

if len(actions) == 0:

    print("- No special action is required.")

else:

    for action in actions:
        print("- " + action)
```

```
# ============================================
# PART 3 — RUN CASE-BASED REASONING
# ============================================

print("\n====================================")
print("CASE-BASED REASONING")
print("====================================")


# The same user input is used as the current CBR problem

current_problem = facts


# RETRIEVE

retrieved_case, score = retrieve(
    current_problem,
    case_base
)


print("\nMost Similar Past Case:")
print(retrieved_case["problem"])

print("\nSimilarity Score:", score)


# REUSE

solution = reuse(retrieved_case)


# REVISE
```

```
final_solution = revise(solution)


# RETAIN

retain(
    case_base,
    current_problem,
    final_solution
)


# ================================================
# FINAL RESULT
# ================================================

print("\n====================================")
print("FINAL RESULT")
print("====================================")

print("\nRule-Based Actions:")

if len(actions) == 0:

    print("- No special action is required.")

else:

    for action in actions:
        print("- " + action)


print("\nCase-Based Solution:")
print("- " + final_solution)

print("\nNew case has been retained successfully.")
```